WELCOME!

You will need groups of 4 for today's activity.

First, try out these Word Ladders. The goal is to transform one word into another by changing one letter at a time, creating a valid word at each step.

COST

GOAL

LEEK

SOUP

WELCOME!

You will need groups of 4 for today's activity.

First, try out these Word Ladders. The goal is to transform one word into another by changing one letter at a time, creating a valid word at each step.

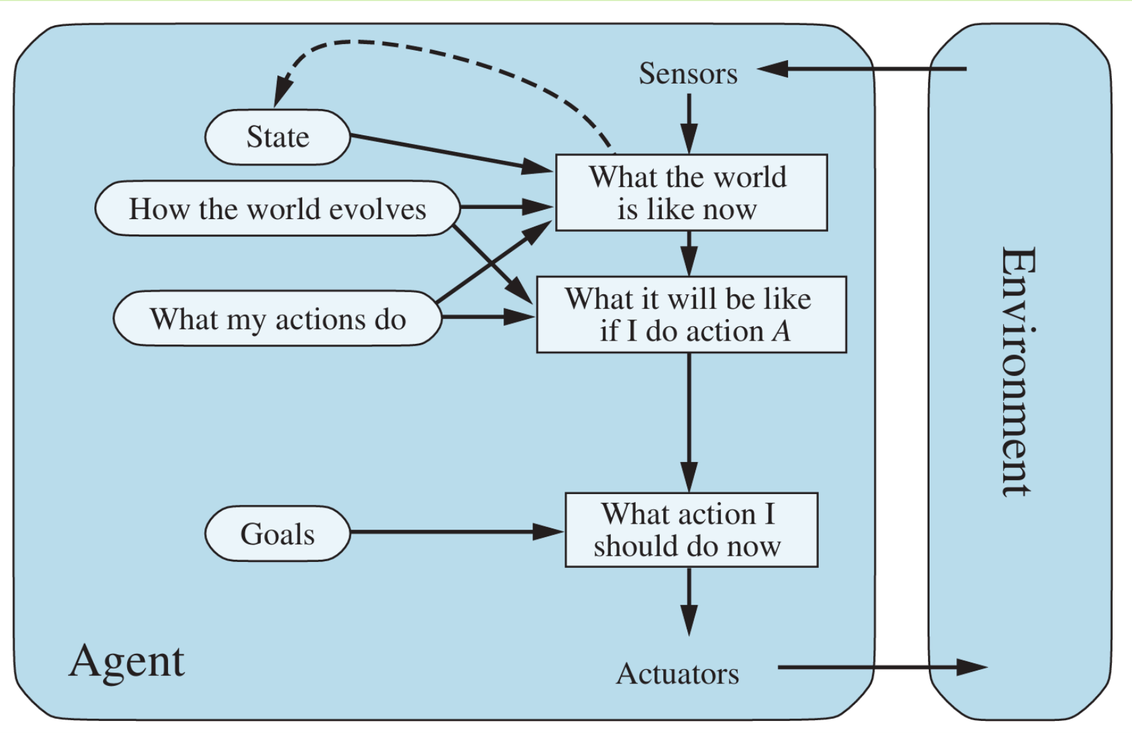
COST
COAT
COAL/GOAT
GOAL

LEEK
SEEK
SEED
SEND
SAND
SAID
SAIL
SOIL
SOUL
SOUP

SEARCH PART 1

SEARCH

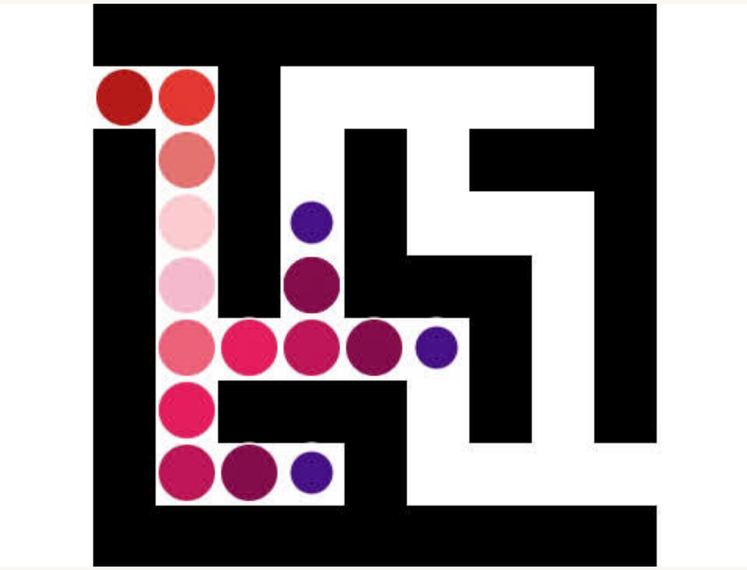
Single agent, in a fully observable, discrete, static, sequential world



Planning Agent

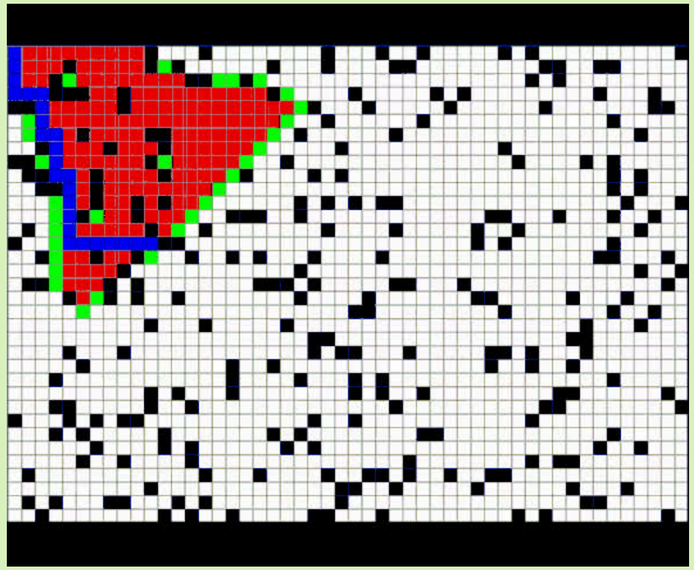
UNINFORMED

- The agent has no additional information other than the goal
- Systematically explores every option



INFORMED

- The agent can estimate how far it is from the goal
- Uses additional information to guide the search process
- Cost-based and goal-directed
- Prioritizing more promising paths, avoids unlikely paths



SEARCH COMPONENTS

STATE
Unique configuration of the world
The relevant information for decision-making

INITIAL STATE
State your agent is in at the beginning of the problem

GOAL STATE(S)
One or more states your agent desires to be in

ACTIONS
Describe state transitions, rules of the game/world

ACTION COST
(optional)

SEARCH COMPONENTS

STATE
Unique configuration of the world
The relevant information for decision-making

GOAT
or
COST

INITIAL STATE
State your agent is in at the beginning of the problem

COST

GOAL STATE(S)
One or more states your agent desires to be in

GOAL

ACTIONS
Describe state transitions, rules of the game/world

change
one letter

ACTION COST
(optional)

SEARCH COMPONENTS

Solution: a sequence of actions which transforms the start state into a goal state

STATE
Unique configuration of the world
The relevant information for decision-making

GOAT or COST

INITIAL STATE
State your agent is in at the beginning of the problem

COST

GOAL STATE(S)
One or more states your agent desires to be in

GOAL

ACTIONS
Describe state transitions, rules of the game/world

change one letter

ACTION COST
(optional)

SEARCH COMPONENTS

Solution: a sequence of actions
which transforms the start state
into a goal state

turn s to a
turn c to g
turn t to l

STATE
Unique configuration of the world
The relevant information for decision-making

GOAT
or
COST

INITIAL STATE
State your agent is in at the beginning of the problem

COST

GOAL STATE(S)
One or more states your agent desires to be in

GOAL

ACTIONS
Describe state transitions, rules of the game/world

change
one letter

ACTION COST
(optional)

GENERIC SEARCH

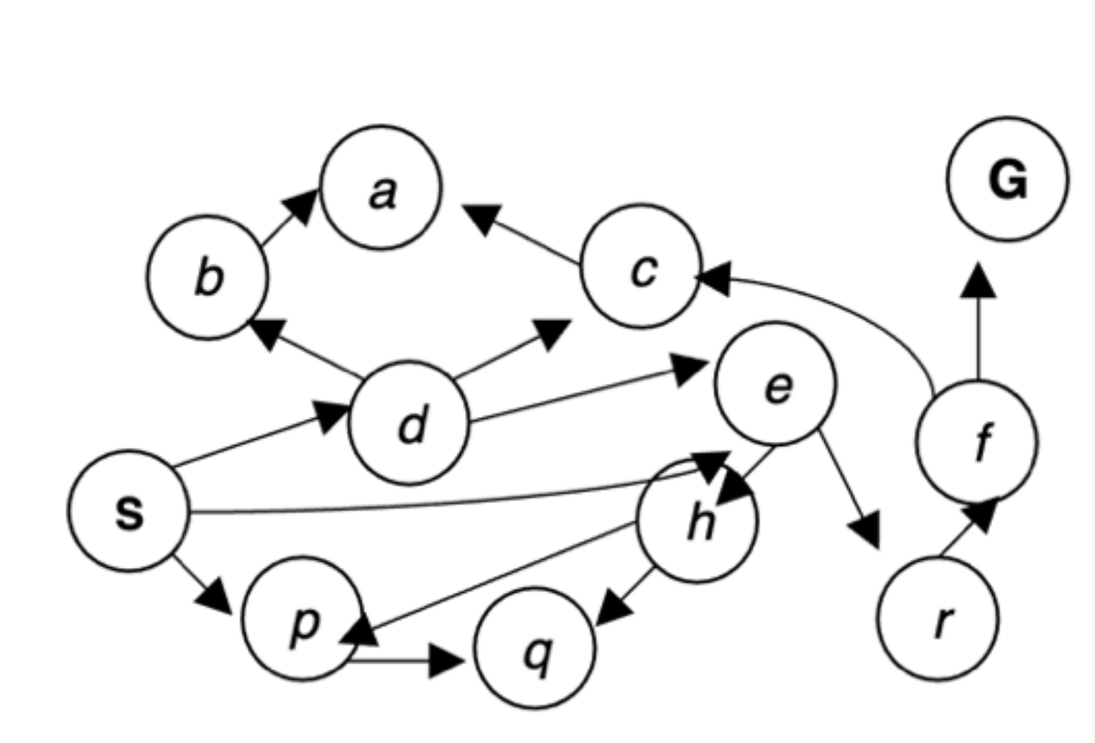
```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

BREADTH FIRST SEARCH



LET'S TRY IT!

GET_SUCCESSORS

You have a dictionary of successors:

A: B, C means B and C are successors of A

Input: a state ID
Output: all successors of that state in the order listed

IS_GOAL

You know the goal state(s)/conditions

Input: a state ID
Output: Yes or No if that state is a goal

OPEN LIST

You keep track of what states we have yet to visit, as well as the parents so we can backtrack the path.

Input: a state ID
Output: the next state to visit, and where that state came from

THE ALGORITHM

You use all the helper functions to create a search tree and find the goal state

Input: the starting state
Output: a list of actions for how to get from the start state to the goal state

WATCH ME

```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

Start Node: S

Goal Node: G

A: none
B: A
C: A
D: B, C, E
E: H, R
F: C, G
G: none
H: P, Q
P: Q
Q: none
R: F
S: D, E, P

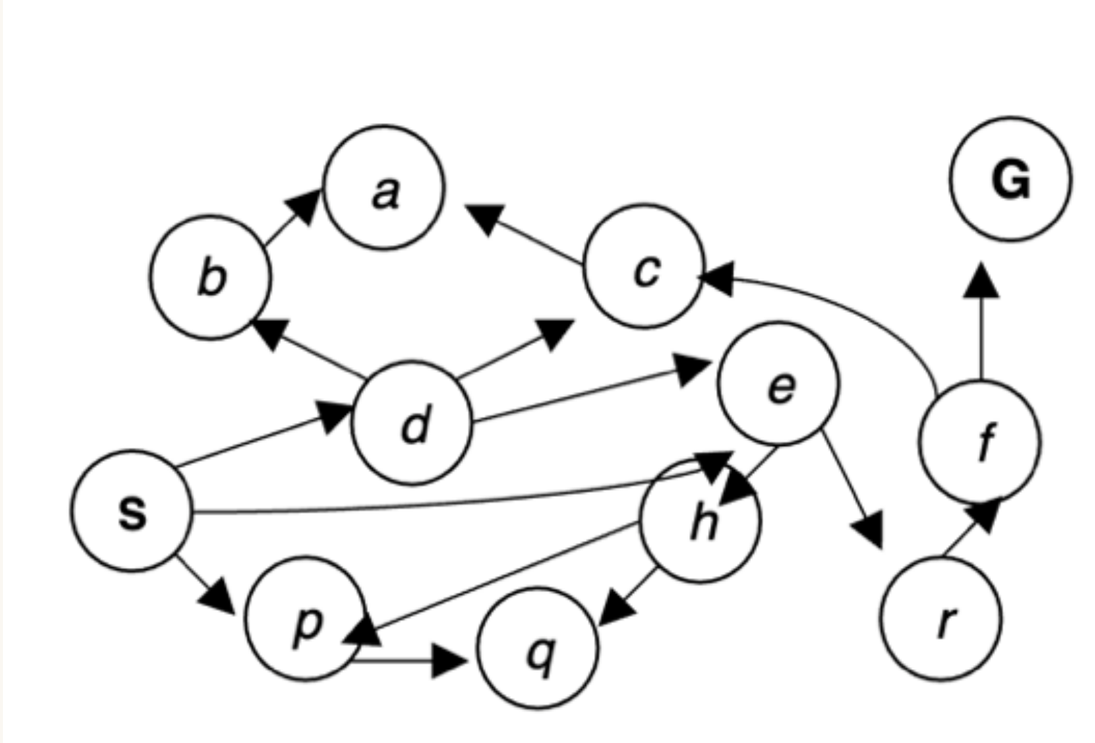
LET'S TRY IT!

PATH:
S → E → R → F → G

VISITED:
S, D, E, P, B, C, H, R, Q, A, F, G

Start Node: S

Goal Node: G



WORLD1

LET'S TRY IT!

Starting Node:

A

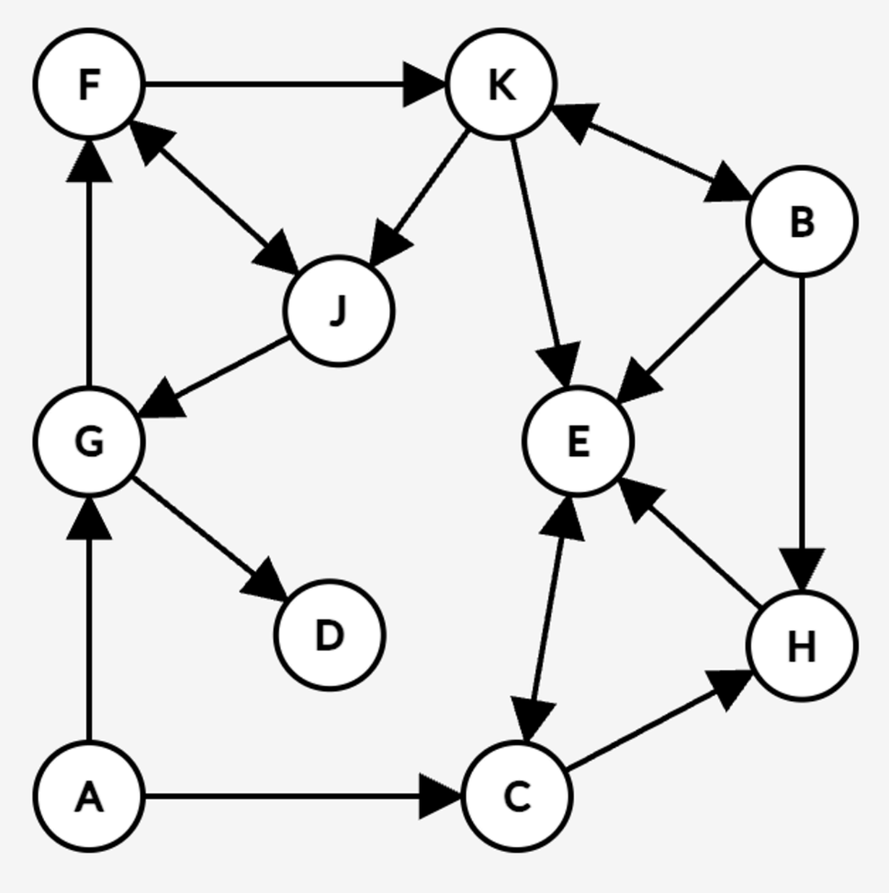
LET'S TRY IT!

PATH:
A → G → F → J

VISITED:
A, C, G, E, H, D, F, J

Start Node: A

Goal Node: J



WORLD2

LET'S TRY IT!

Starting Node:

3

Remember: a solution is a sequence of actions which transforms the start state into a goal state

WORLD3

LET'S TRY IT!

PATH:

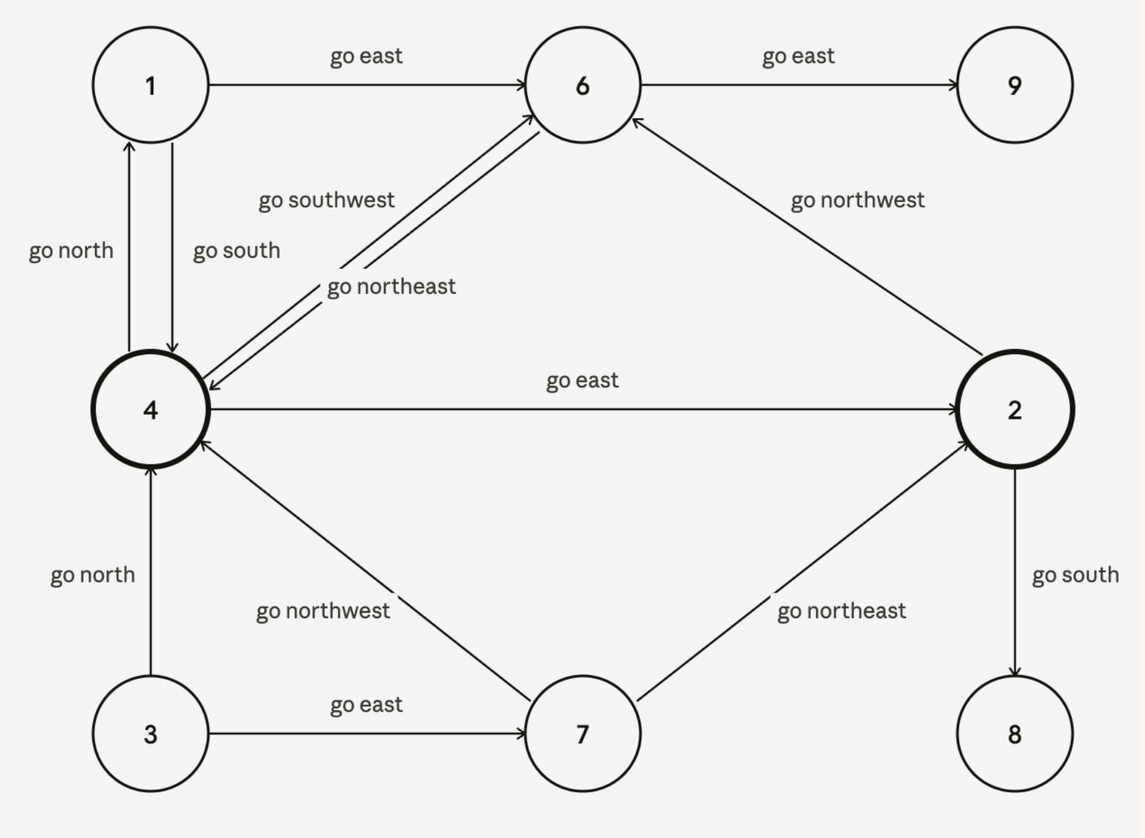
go north → go northeast → go east

VISITED:

3, 4, 7, 1, 2, 6, 8, 9

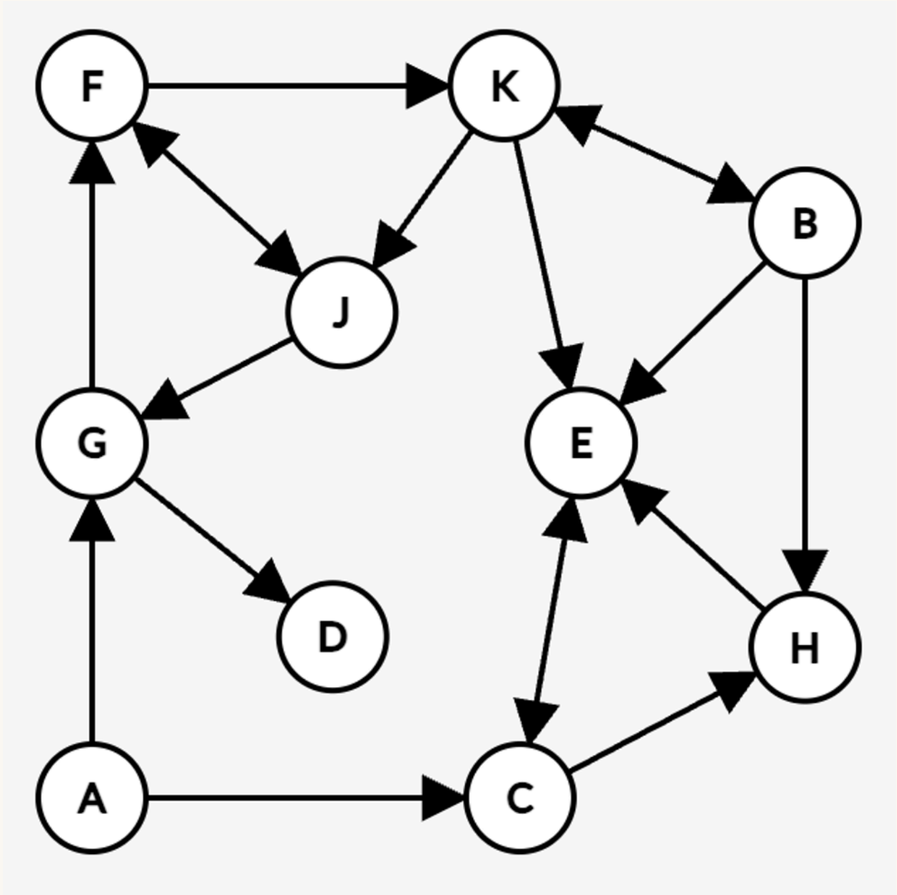
Start Node: 3

Goal Node: 9



WORLD3

DEPTH FIRST SEARCH




```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

Start Node: A

Goal Node: J

A: C, G
B: E, H, K
C: E, H
D: none
E: C
F: J, K
G: D, F
H: E
J: G
K: B, E, J

DEPTH FIRST SEARCH

```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

Start Node: A

Goal Node: J

A: C, G
B: E, H, K
C: E, H
D: none
E: C
F: J, K
G: D, F
H: E
J: G
K: B, E, J

LET'S TRY IT!

Starting Node:

B

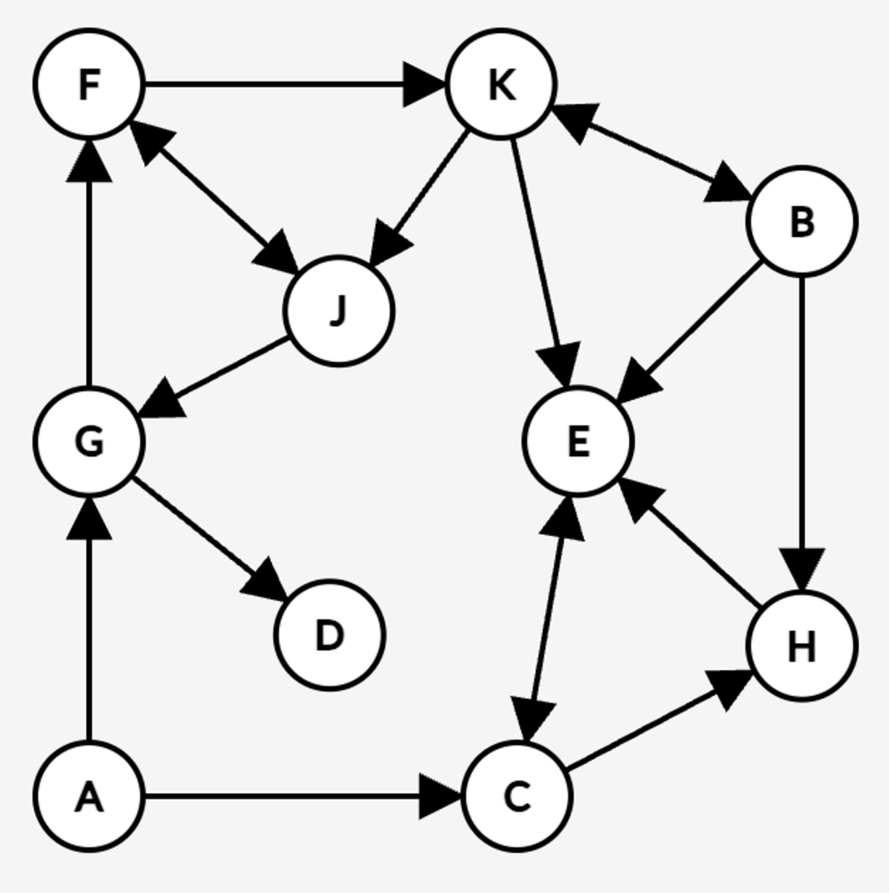
LET'S TRY IT!

PATH:
B → K → J → G → D

VISITED:
B, K, J, G, F, D

Start Node: B

Goal Node: D



WORLD4

LET'S TRY IT!

Starting Node:

5

LET'S TRY IT!

PATH:

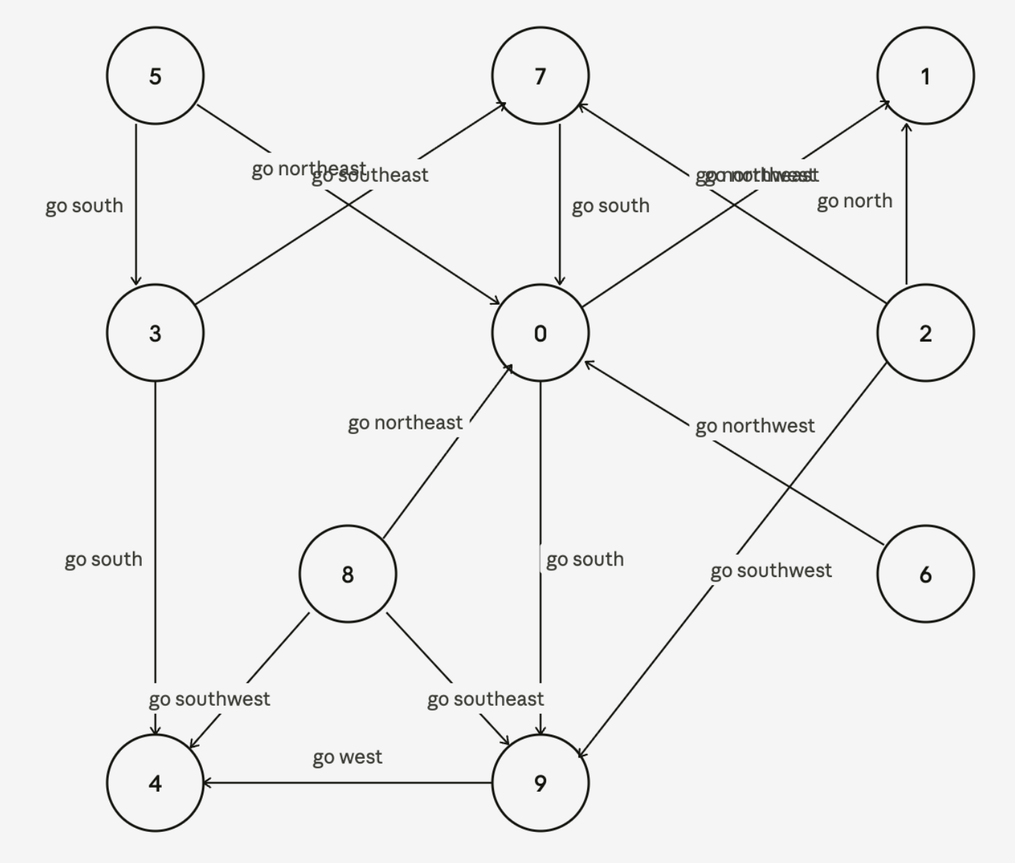
go south → go northeast → go south → go northeast

VISITED:

5, 3, 7, 0, 9, 4, 1

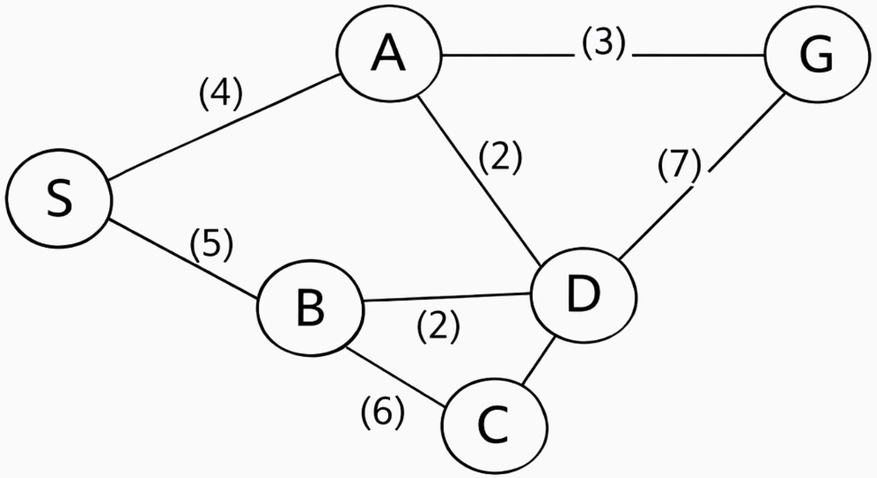
Start Node: 5

Goal Node: 1



WORLD5

UNIFORM COST SEARCH




```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

Start Node: S

Goal Node: G

A: (D, 2), (G, 3), (S, 4)
B: (C, 6), (D, 2), (S, 5)
C: (B, 6), (D, 1)
D: (A, 2), (B, 2), (C, 1), (G, 7)
G: (A, 3), (G, 7)
S: (A, 4), (B, 5)

UNIFORM COST SEARCH

```
generic_search(init_state, goal, actions=[a1, a2, ..., an]):
    closed = []
    open = [init_state]
    current = init_state

    while not is_goal(current) and open is not empty:
        add current to closed
        sucessors = get_successors(current, actions) that are not in closed
        insert successors into open
        current = open.pop()


    if is_goal(current):
        plan = reconstruct path
        return plan

    return FAIL
```

Start Node: S

Goal Node: G

A: (D, 2), (G, 3), (S, 4)
B: (C, 6), (D, 2), (S, 5)
C: (B, 6), (D, 1)
D: (A, 2), (B, 2), (C, 1), (G, 7)
G: (A, 3), (G, 7)
S: (A, 4), (B, 5)

LET'S TRY IT!

Starting Node:

A

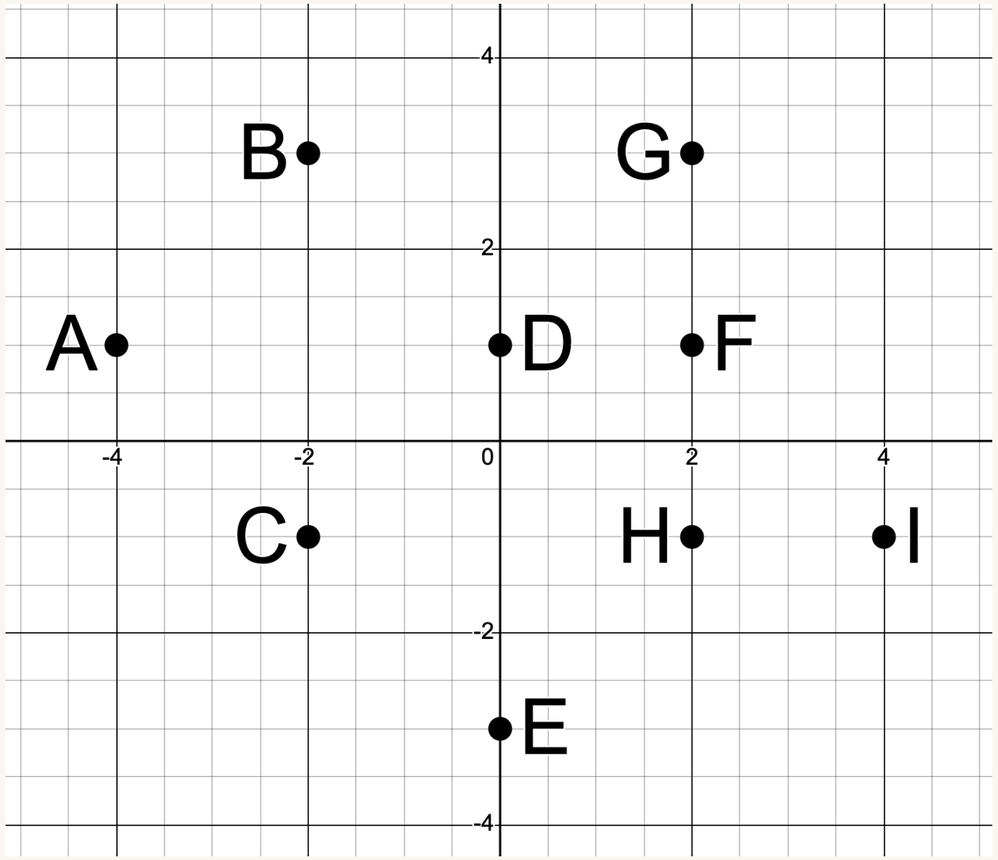
LET'S TRY IT!

PATH:
A → C → E → I

VISITED:
A, C, B, E, D, I

Start Node: A

Goal Node: I



WORLD6

LET'S TRY IT!

Starting Node:

C

LET'S TRY IT!

PATH:

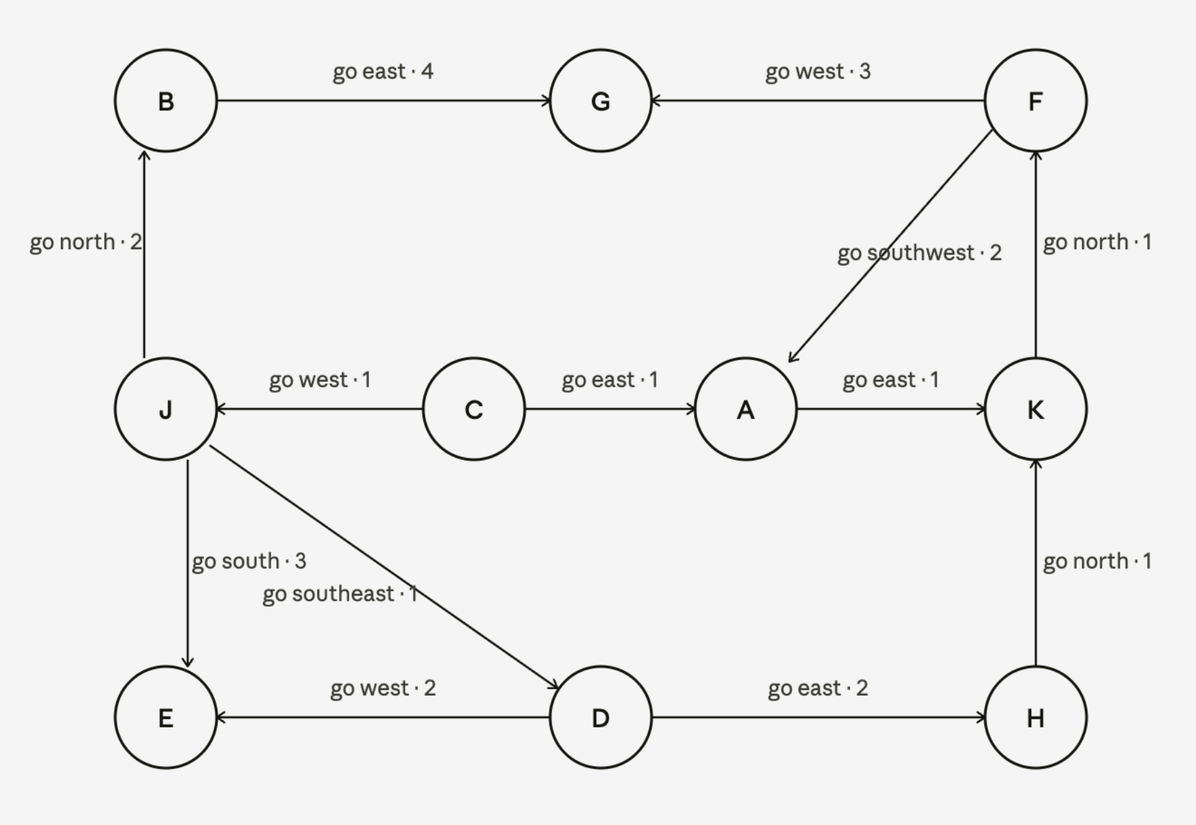
go east → go east →
go north → go west

VISITED:

C, A, J, K, D, B, F, H, E, G

Start Node: C

Goal Node: G

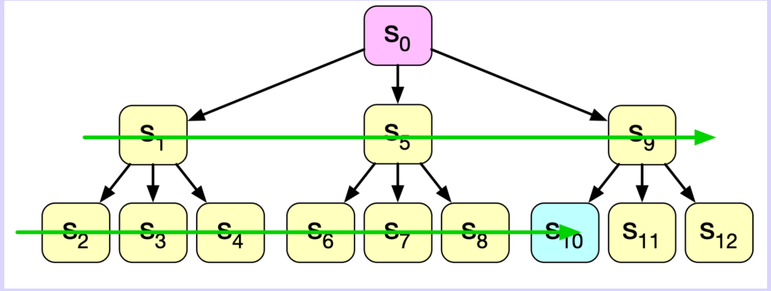


WORLD7

RECAP

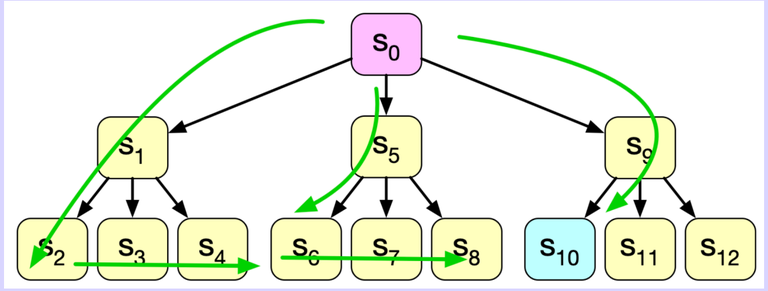
01 BREADTH-FIRST SEARCH

Strategy: shallowest unexpanded node
Implementation: FIFO



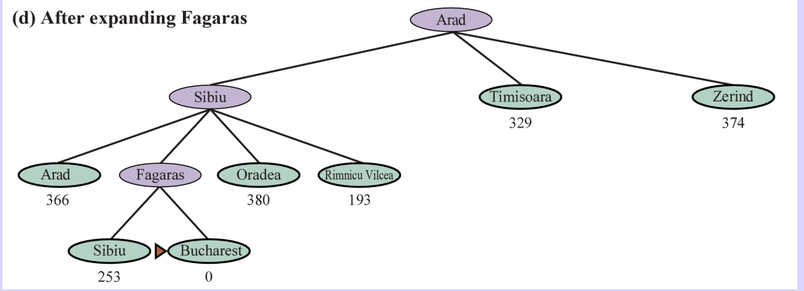
02 DEPTH-FIRST SEARCH

Strategy: expand the deepest unexpanded node
Implementation: LIFO



03 UNIFORM COST SEARCH

Strategy: expand the cheapest unexpanded node
Implementation: Priority queue of cumulative cost



THOUGHTS?

1. How did your group organize information across the roles?
2. Was there a role that was easier/harder than expected?
3. In what ways was your role limited?
4. How did changing the worlds change the group's behavior?
5. How did changing the Frontier's data structure change the groups behavior?
6. If you had to explain this activity to a student who missed class, how would you summarize its main goal and what you learned from it?